Bloodline 2.0 — Mule Network Detection and Tracing

Bloodline is a prototype decision-support system for banks and payment providers. It combines transaction risk scoring, account-level mule indicators, graph-based investigation, estimated fund-flow tracing, and human-reviewed case workflows.

> Prototype boundary: This project is for demonstration and evaluation on synthetic data. It is not production banking software, legal advice, or an automated fraud-classification system. A risk score is an investigation signal, not a finding that a person or account committed fraud.

Project goals

• Score a payment before it is released and return a decision recommendation with evidence-based reason codes.

• Identify suspicious account relationships and money movement patterns over time.

• Help investigators follow an origin payment through splitting, layering, and merging transfers.

• Rank possible next-hop accounts to support bank-controlled intervention.

• Keep analyst decisions, customer outcomes, approvals, and case events auditable.

• Evaluate performance against a simple rule-only baseline using disclosed synthetic ground truth.

Core capabilities

1. Payment and payee risk checks — transaction risk, account mule score, rule/model reasons, and an allow, delay, or review recommendation.

2. Mule-network analysis — fan-in/fan-out, rapid pass-through, structuring, cycles, dormant-account activation, and shared device/IP/phone links.

3. Money DNA — estimated proportional tracing from a selected origin transaction, represented as a time-aware flow diagram and replayable network.

4. Next-hop ranking — top candidate destination accounts with probabilities and evidence; use an edge-frequency/risk fallback when the model is unavailable.

5. Cooling-off and review queue — configurable delay or hold proposals, countdowns, reasons, and recorded customer outcomes.

6. Case Clock — configurable case states, deadlines, draft documents, approval gates, and an append-only audit history.

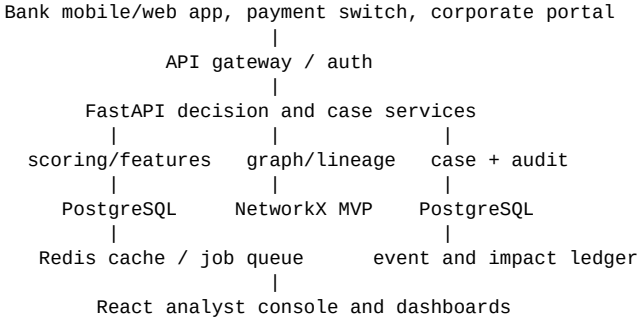
7. Analyst feedback — record confirmed fraud/false-positive outcomes; adjust thresholds in a bounded and observable way before considering model retraining.

8. Business Shield — check vendor payout destination risk and recent bank-detail changes.

9. Impact ledger — prevented, at-risk, and lost amounts; false-positive cost; and comparison with a documented baseline.

10. Evidence summaries — generate a structured case summary from validated evidence. Any LLM output is advisory, schema-checked, and never the source of a score, legal determination, or approval.

Suggested architecture



Start with a modular monolith and a single-bank view. Keep interfaces clear so a graph database, streaming ingestion, or additional bank integrations can be introduced later. Do not imply that the prototype can see accounts or transactions at other banks.

Graph and visual structures

1. Temporal heterogeneous transaction graph

Represent data as a directed, time-aware multigraph. Multiple transfers can connect the same accounts, so transfer edges must retain their own IDs and timestamps.

Node types: account, customer/entity, device, phone, IP address, payee/vendor, case, and honeypot marker. Use pseudonymous IDs in the prototype.

Edge types: `TRANSFERRED_TO` (directed; amount, currency, timestamp, channel, status), `OWNS`/`CONTROLS`, `USED_DEVICE`, `USED_PHONE`, `USED_IP`, `PAID_VENDOR`, and `LINKED_TO_CASE`. Shared-attribute links are investigative context and should not be treated as proof of common control.

Account attributes: account age, risk score, velocity features, status, institution scope, and data-quality/provenance metadata. Transfer attributes: amount, time, channel, payment status, and lineage estimate. Keep raw sensitive identifiers out of graph labels and UI.

2. Money DNA flow graph (Sankey)

For a selected origin transfer, build a directed acyclic display graph from time-filtered outgoing transfers. Assign layers by hop count; represent splits and merges; size bands by estimated tainted amount; show frozen or in-transit values separately. Add a timeline scrubber that updates both this flow view and the transaction graph. Label the view estimated lineage and make the tracing method and time window inspectable. Proportional tracing is an attribution assumption, not a legal finding.

3. Investigation network view

Show accounts and related attributes with distinct node styles, transfer direction, amount/time tooltips, risk indicators, and filters for time, amount, channel, and edge type. Support expanding one or two hops, focusing on an origin, and opening an evidence panel. Avoid visual encodings that imply guilt from a score alone.

4. Next-hop prediction overlay

Use dashed edges for predicted destinations, with rank, probability, and evidence. After an authorized action is recorded, display the actual event distinctly. Prediction and actual-transfer edges must never be visually conflated.

5. Case workflow state machine

Suggested configurable states: `suspected -> show_cause_pending -> representation_window -> order_pending -> classified -> reported/closed`; allow an explicit false-positive closure path. Enforce role-based approval and retain actor, timestamp, prior state, new state, and reason. Do not hard-code legal durations; load versioned policy configuration and source metadata.

API outline

Use versioned JSON REST APIs with OpenAPI docs, bank-scoped authorization, idempotency keys for write operations, correlation IDs, and audit logging. Example routes:

Method and route	Purpose
`POST /api/v1/score`	Score a transaction; return score, verdict recommendation, reason codes, model/rule versions, and latency metadata.
`POST /api/v1/payee-check`	Pre-payment check for a beneficiary account/payee.
`POST /api/v1/vendor-check`	Check vendor destination risk and bank-detail-change signals.
`GET /api/v1/accounts/{account_id}/risk`	Retrieve current account risk summary and evidence references.
`GET /api/v1/accounts/{account_id}/network`	Retrieve a filtered, bounded neighborhood graph.
`POST /api/v1/lineage`	Build estimated lineage for an origin transaction and time window.
`POST /api/v1/predictions/next-hop`	Return ranked candidate next-hop accounts with calibrated confidence where available.
`POST /api/v1/payments/{payment_id}/outcomes`	Record cancelled, proceeded, held, released, or other bank-defined outcome.
`POST /api/v1/cases` / `GET /api/v1/cases/{case_id}`	Create or retrieve an investigation case.
`POST /api/v1/cases/{case_id}/transitions`	Request a state transition; enforce authorization and approval requirements.
`POST /api/v1/cases/{case_id}/decisions`	Record analyst decision, label, rationale, and evidence references.
`GET /api/v1/impact`	Return status-ledger totals and baseline comparison for a time range.
`GET /health` / `GET /ready`	Liveness and readiness probes without exposing sensitive data.

Example score response

```
{
  "transaction_id": "txn_demo_001",
  "risk_score": 0.91,
  "recommended_action": "DELAY_REVIEW",
  "reason_codes": [
    {"code": "NEW_PAYEE_HIGH_VELOCITY", "evidence": "14 inbound senders in 24h"},
    {"code": "RAPID_PASS_THROUGH", "evidence": "recent inbound funds followed by outbound transfer"}
  ],
  "model_version": "demo-2026-01",
  "policy_version": "synthetic-demo-v1",
  "decision_id": "dec_demo_001"
}
```

Do not return a bare score without its scope, version, reasons, and recommendation. Document each field, enum, validation rule, error, timeout, and authorization requirement in the OpenAPI schema.

Suggested technology choices

Area	Prototype choice	When to upgrade
Editor and source control	VS Code, Git, GitHub/GitLab	Add branch review and CI as collaborators join.
Backend/API	Python 3.12+, FastAPI, Pydantic, Uvicorn	Split services only when load or ownership requires it.
Data	PostgreSQL; SQLAlchemy/Alembic	Add warehouse/lakehouse for larger historical analytics.
Graph computation	NetworkX for synthetic-data MVP	Evaluate Neo4j for interactive persisted graph queries; compare performance on representative data.
Cache/jobs	Redis and a task queue if needed	Kafka or another event stream for high-volume real-time ingestion.
ML	pandas/Polars, scikit-learn, XGBoost, SHAP	Add experiment/model registry and drift monitoring before operational use.
Frontend	React + TypeScript; D3/Cytoscape.js for network; Sankey library for flows	Streamlit is a quick alternative for a short hackathon demo.
API exploration	OpenAPI/Swagger UI, Postman or Bruno	Add contract checks to CI.
Packaging	Docker Compose	Kubernetes only for a real deployment need.
Observability	Structured logs, OpenTelemetry, Prometheus/Grafana	Integrate with the bank's approved monitoring stack.

For a compact demo, use FastAPI + PostgreSQL + NetworkX + XGBoost/scikit-learn + React/TypeScript + Docker Compose. Pick either React or Streamlit for the first UI rather than building both.

Data and model safety

• Use generated or properly approved de-identified data. The demo generator should include mule chains, legitimate high-velocity look-alikes, noise, honeypot events, and ground-truth lineage.

• Keep identity resolution and real customer identifiers outside the demo graph. Apply data minimization, retention limits, encryption, role-based access, and secrets management for any approved real-data environment.

• Treat shared phone/device/IP signals as uncertain associations; display provenance and confidence.

• Record model, feature, rules, and policy versions for each decision. Preserve the evidence snapshot used at decision time.

• Calibrate thresholds on a held-out set; report precision, recall, false-positive rate, latency, and subgroup/fairness checks where appropriate.

• Never silently retrain or let analyst feedback directly convert a person/account into a fraud classification. Use approved labels, review, and controlled model releases.

• Make holds/freezes bank-configured actions behind authorization and audit controls. A prototype should simulate these actions unless connected to an explicitly authorized environment.

• Store regulatory rules as versioned configuration with official source links, effective dates, applicability, and reviewer. Confirm current entity-specific requirements with qualified compliance/legal staff before use. The source overview is not legal authority.

Evaluation plan

Report results on synthetic held-out scenarios and compare against a documented rule-only baseline:

• Detection: precision, recall, false-positive rate, PR-AUC where useful.

• Timing: time from suspicious pattern onset to flag; compare with baseline.

• Next hop: top-1/top-3 accuracy and amount intercepted before a simulated exit.

• Lineage: estimated amount and path agreement against generator ground truth.

• Operations: review queue volume, decision latency, and case-clock simulation completion.

• Business Shield: attack detection and legitimate invoice hold rate.

• Impact: prevented/at-risk/lost accounting rules, with deduplication and reversal behavior documented.

State clearly that all demo metrics are synthetic and do not predict live bank performance.

Local development

Prerequisites: Python, Node.js, Git, and Docker Desktop (optional for local services).

```
git clone <your-repository-url>
cd <repository-directory>
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

If using the React UI, install its dependencies from the `frontend/` directory and start its development server. Add exact commands after the repository structure and scripts are established.

Suggested repository layout

```
app/
  api/              # route definitions and API schemas
  services/         # scoring, graph, lineage, cases, impact
  models/           # feature/model loading and version metadata
  policies/         # versioned demo rules and source metadata
  db/               # models, migrations, repositories
  main.py
frontend/
  src/              # payment demo and analyst console
data/               # generator only; avoid committing sensitive data
notebooks/          # exploratory work, not production logic
tests/              # unit, API, graph, and scenario tests
docs/               # architecture, threat model, data dictionary
docker-compose.yml
requirements.txt
README.md
```

README sections to keep current

This file should explain the project purpose and prototype boundary; key features; architecture and graph semantics; setup and run commands; environment variables (names only, never secret values); API links/examples; synthetic-data generation; model and graph assumptions; evaluation methodology and baseline; screenshots/demo flow; security and privacy assumptions; regulatory-source/versioning policy; limitations; contribution workflow; license; and contact/maintainer details.

Demo walkthrough

1. Generate synthetic data and show a legitimate account beside a suspicious mule chain.

2. Submit a payment/payee check and explain the reasons returned by the API.

3. Display a configured delay/review recommendation and record a simulated customer outcome.

4. Open the origin in the network and Money DNA views; replay transfers over time.

5. Inspect next-hop candidates and evidence; simulate an authorized hold/freeze.

6. Create a case, show pending human approvals and configurable clocks, then record a false-positive or confirmed-fraud label.

7. Show the impact ledger and metrics beside the rule-only baseline.

Limitations and next steps

This prototype has a single-bank data view, synthetic evaluation, simplified attribution, and no validated production integrations. Next steps are to define an event/data contract with a bank sponsor, threat-model the service, validate regulatory configuration with compliance, test graph/model behavior on approved representative data, and conduct latency, resilience, privacy, and fairness reviews before any pilot.

License

Choose and add a license before distributing the repository.